INTERNATIONAL CAMPUS OF SCIENCE AND TECHNOLOGY

Faculty of Computing

Department of Information Technology

S Y S T E M D O C U M E N T A T I O N

Sri Lanka Smart Tourism & AI Trip Planner

An AI-Powered Personalised Travel Planning System for Sri Lankan Tourism

Team GenZ Minds

M.M. Leenath (03241049)

M.T. Nihma (03241195)

M.N.N. Naajee (03241014)

F.F. Mufasa (03241020)

M. Ameeth (03241086)

Document Control

Document details

Item	Detail
Document title	System Documentation: Sri Lanka Smart Tourism & AI Trip Planner
Project	An AI-Powered Personalised Travel Planning System for Sri Lankan Tourism
Team	GenZ Minds
Institution	International Campus of Science and Technology, Faculty of Computing, Department of Information Technology

Purpose of this document

This document describes the system as built, as distinct from the project proposal, which describes the system as intended. Every figure quoted here was taken from the running implementation or from the test suites, not from the proposal's expectations.

How to read this document

Sections 1 to 4 present the architecture, technology stack and data layer. Sections 5 and 6 describe the two components that carry the intellectual weight of the system: the price model and the itinerary engine. Sections 7 to 10 cover the API, security, frontend and third-party integrations. Sections 11 to 13 cover testing, known limitations and deployment.

Table of Contents

Document Control 2
Purpose of this document 2
How to read this document 2
Table of Contents 3
List of Tables 4
1. System Overview 5
2. Architecture 6
2.1 Logical Tiers 6
2.2 Request Flow 6
2.3 Project Modules 6
3. Technology Stack 7
Choice of SQLite 7
4. Data Layer 8
4.1 The Accommodation Dataset 8
4.2 SQLite Schema 9
5. Machine Learning Subsystem 11
5.1 Objective and Target 11
5.2 Preprocessing and Split 11
5.3 Results 11
5.4 Interpretation 12
5.5 Target Leakage: A Required Disclosure 12
6. Itinerary Generation Engine 13
7. API Reference 15
7.1 Dataset and Model 15
7.2 Authentication 15
7.3 Destinations 15
7.4 Trips 15
7.5 Admin 16
7.6 Status Code Conventions 16
8. Authentication and Authorisation 17
9. Frontend Implementation 18
10. External Integrations 19
11. Testing and Verification 20
12. Known Limitations 21
13. Deployment and Runbook 22
Appendix A: Summary of Design Decisions 23
Appendix B: Sample Generated Trip 24

List of Tables

Table 1 Project Modules and Responsibilities
Table 2 Dataset Column Profile
Table 3 Accommodation Types by Frequency
Table 4 SQLite Tables
Table 5 Machine Learning Model Comparison
Table 6 Top Feature Importances
Table 7 Itinerary Routes by Trip Length
Table 8 Notes Keyword Rule Families
Table 9 Dataset and Model Endpoints
Table 10 Authentication Endpoints
Table 11 Destination Endpoints
Table 12 Trip Endpoints
Table 13 Admin Endpoints
Table 14 API Status Code Conventions
Table 15 Frontend Routes
Table 16 Test Suite Results
Table 17 Known Limitations
Table 18 Registry Column Profile

1. System Overview

The system is a three-tier web application that plans Sri Lankan itineraries from a real accommodation dataset and quotes overnight costs using published nightly rates. Three distinguishing properties shape every design decision described in this document.

Deterministic itineraries. The same planner form always produces the same plan. The system makes no language-model call and uses no random seed, so a plan can be regenerated, stored, shared by link and re-read months later with identical content.

Real quoted costs. Overnight stays are selected from the 2,130-row priced dataset, so every accommodation figure in a budget breakdown is a published price rather than an estimate.

An honestly framed price model. The dataset's prices are derived from a 25-row rate card. The model is therefore documented and presented as a price estimator and rate-card recovery tool, not as evidence of predictive accuracy. This is discussed in full in Section 5.

The system is scoped to search and planning. It deliberately performs no booking and handles no payment; third-party availability is handed off to Booking.com through an affiliate link.

2. Architecture

2.1 Logical Tiers

The system follows a conventional three-tier architecture.

Presentation tier. A single-page React 18 application built with Vite. It owns all rendering, form state and routing, and communicates with the application tier over JSON.

Application tier. A Flask 3.0.3 application. Dataset and model routes live in the application factory; every route that mutates state is organised into one of four Flask blueprints.

Data tier. Three kinds of storage with clearly separated roles: SQLite holds mutable state, CSV holds the read-only dataset, and joblib holds the trained models.

2.2 Request Flow

A typical request travels from the browser to the Vite development server, which proxies /api to Flask. The Flask router then dispatches it to either a blueprint guard or a public handler, which in turn calls the store or the model. The resulting JSON response is shaped by a serializer for the user interface. The separation between public and authenticated handlers is enforced in the route layer rather than in the interface, so a hidden button is never the only protection.

2.3 Project Modules

Table 1: Project Modules and Responsibilities

The API was split into blueprints, rather than grown inside a single module, once the seventeenth endpoint was added. Each blueprint owns one resource and imports the store directly, which keeps import order simple and avoids circular dependencies: the dataset reaches the blueprints through the application configuration rather than by import.

3. Technology Stack

Runtime components and versions tested

Module	Tier	Responsibility
app.py	Application	App factory, dataset load, model routes
store.py	Data	SQLite schema, CRUD, admin aggregates
auth.py	Application	HMAC tokens and route guards
routes_auth.py	Application	Register, login, profile, demo
routes_destinations.py	Application	Catalogue read and admin CRUD
routes_trips.py	Application	Plan, list, read, delete trips
routes_admin.py	Application	Stats, analytics, user management
itinerary.py	Application	Route, attractions, stays, budget
train.py	ML	Trains both models, writes artifacts
import_data.py	Data	Priced workbook to CSV projection
import_registry.py	Data	Expanded workbook to registry CSV

Choice of SQLite

The proposal named MySQL or PostgreSQL. No client for either is installed on the development machine, whereas sqlite3 ships inside Python and needs no server, credentials or service to start. The store module is the only place that touches the database driver, so moving to a server database is confined to that file and a connection string.

4. Data Layer

The data tier holds two unrelated kinds of data, and it is important to keep them distinct. The accommodation dataset is a fixed, read-only table of published rates. The application state is mutable data created by the application itself.

4.1 The Accommodation Dataset

The source workbook contains two sheets. Sheet 1 is the accommodation register and Sheet 2 is the price list from which Sheet 1 was generated. The script import_data.py provides the reproducible projection from workbook to CSV and is the single source of truth for both derived files.

Table 2: Dataset Column Profile

Column	Populated	Distinct
Type	100.0%	11
Name	100.0%	2120
Address	100.0%	2095
Rooms	100.0%	128
Grade	86.2%	11
District	100.0%	25
AGA Division	99.1%	183
PS/MC/UC	99.9%	232
Longitude	64.2%	1358
Latitude	64.3%	1359
Price Single (LKR)	100.0%	19
Price Double (LKR)	100.0%	18
Price Triple (LKR)	100.0%	18
Meal Plan	100.0%	2
Price Updated On	100.0%	1
Remarks	100.0%	1

Two columns deserve comment. Grade is blank for 293 rows; the pipeline treats the blank as a genuine category rather than dropping the rows, so no accommodation is lost from search results. Longitude and Latitude are missing for roughly 760 rows; these are imputed with district medians so that map markers still appear. This approximation is recorded as a limitation in Section 12.

The Remarks column carries no information: it holds the same constant string, “Estimated rate - verify”, in all 2,130 rows. This single fact rules out any natural language processing component, as explained in Section 6.5.

Table 3: Accommodation Types by Frequency

Type	Count
Guest Houses	894
Home Stay Units	378
Bangalows	348
Tourist Hotels	229
Classified Hotels (1-5 Star)	141
Rented Apartments	64
Boutique Villas	36
Boutique Hotels	28

Rented Homes	6
Heritage Bungalows	4
Heritage Homes	2

Price distribution. The double-occupancy rate, which is the model's target, spans LKR 5,000 to LKR 72,000, with a median of LKR 10,000 and a mean of LKR 13,182. There are only 18 distinct double-rate values across 2,130 rows, which is the first visible indication that the prices are generated rather than observed. Rooms range from 1 to 541, with a median of 6. The meal plan is bed and breakfast for 2,063 rows and half board for 67.

Ten property names are duplicated in the source data. They are retained rather than deduplicated because removing rows would change the price distribution to which the model is fitted, and the duplicates are legitimate, separate registrations.

4.2 SQLite Schema

Three tables hold all mutable state. The database file is created automatically on first run and is seeded with a demo account and a nine-entry destination catalogue.

Table 4: SQLite Tables

Table	Purpose	Notable columns
users	Accounts	uid, email, password_hash, role
destinations	Catalogue	uid, name, category, featured
trips	Saved plans	uid, user_uid, payload, duration

The trips table stores the whole generated plan as a JSON payload rather than normalising days and activities into separate tables. This is a deliberate choice: a trip document is written once, read whole and never partially updated, so normalisation would add joins and gain nothing. Two indexes support the queries that exist, one on the trip owner and one on the featured flag.

Connections are opened in WAL mode with foreign keys enforced. A context manager commits on success and rolls back on failure, so a failed write cannot leave a half-written trip behind.

Identifier design. Primary keys are internal integers. Every row also carries a twelve-character public uid that appears in URLs and cannot be guessed sequentially. The store's public serializers remove the password hash and the internal key before any row reaches a response.

4.3 The Establishment Registry

The registry is a second catalogue, kept alongside the priced accommodation table rather than replacing it. It holds 4,744 establishments across all 25 districts, compiled on 5 October 2026 from the SLTDA accommodation, restaurant and eating-place directories, Hostelworld city listings and a historical open-data mirror. The 2,130 records in the priced table are a subset of this compilation, so the two sources agree rather than conflict.

Where the priced table answers what does a night cost, the registry answers what else is there. It carries contact details, official grades, star ratings, licence status, coordinate provenance and source URLs. It has no price columns, so it is never used for pricing, stay selection or budgeting; those continue to read the priced table. Offering a price filter here would imply a meaning the data does not have.

Table 18

Three readings are worth recording. First, 1,180 records are marked historical registry only, meaning the source cannot confirm the business still operates; the interface shows this rather than hiding it. Second, 1,698 rows carry an expired licence. The workbook's own metadata is explicit that an expired licence does not prove closure, so the interface labels licence status instead of filtering on it. Third, 984 records have no coordinate at all and others are marked withheld or needing review, so their map pins are imputed from district medians and the honest coordinate status travels with the record.
The registry is served through its own endpoints, GET /api/registry and GET /api/registry/<id>, with search and type, district, grade and licence filters. It is reachable from the Registry link in the navigation bar.

Column	Populated	Distinct
District	100.0%	25
Name	100.0%	4712
Type	100.0%	15
Source Type	100.0%	27
Rooms	66.3%	146
Grade	76.8%	6
Latitude	74.1%	3180
Longitude	74.1%	3097
Phone	52.3%	2403
Mobile	28.5%	1287
Email	62.6%	2962
Website	22.1%	906
Registration Number	41.2%	1954
Licence Status	58.9%	25
Record Status	100.0%	4
District Basis	100.0%	9
Coordinate Status	100.0%	32
Source Vintage	100.0%	3
Primary Source URL	100.0%	851
Notes	36.6%	365

5. Machine Learning Subsystem

5.1 Objective and Target

The model predicts Price Per Night – Double (LKR), the double-occupancy nightly rate, from property characteristics. It is presented to users as a price estimator on the accommodation detail page and within a standalone estimator widget.

A Random Forest is the primary model, and a single Decision Tree is trained alongside it as an interpretable baseline. Both are trained by train.py, which writes one scikit-learn pipeline per model together with a shared metrics file.

5.2 Preprocessing and Split

Categorical columns are one-hot encoded and numeric columns are left unchanged, all inside a single scikit-learn pipeline. The preprocessing learned at training time is therefore applied automatically at prediction time and cannot drift from it. The data is split 80/20, giving 1,704 training rows and 426 test rows out of 2,130. The Random Forest was tuned to 200 estimators with unlimited depth and a minimum of one sample per leaf.

5.3 Results

Table 5: Machine Learning Model Comparison

Metric	Random Forest	Decision Tree
RMSE (LKR)	83.51	0.00
MAE (LKR)	8.77	0.00
R squared	0.99992	1.00000
MAPE (%)	0.0274	0.0000

For context, a trivial predictor that always guessed the mean price would have a mean absolute error of LKR 5,872.77. Both models perform far better than that baseline, but the comparison is not the main point; Section 5.5 explains why.

Table 6: Top Feature Importances

Feature	Importance
Meal Plan HB	0.3145
Meal Plan BB	0.2921
Type: Boutique Villas	0.1275
Grade: DELUXE	0.1215
Type: Home Stay Units	0.0308
Grade: FIVE	0.0303
Grade: THREE	0.0188
Grade: SUPERIOR	0.0138

Meal plan alone accounts for sixty percent of the total importance, followed by property type and grade. Location contributes almost nothing, which is itself a finding: once type and grade are known, the price is already determined.

5.4 Interpretation

The model recovers the rate card accurately and generalises across the one-hot encoding without overfitting artefacts. It is useful for answering questions such as what a deluxe property in a given district should cost, and for flagging a listed rate that sits far from its peers.

5.5 Target Leakage: A Required Disclosure

This section exists because the headline accuracy is misleading if read naively. The project states the reason plainly rather than presenting an inflated figure.

Sheet 1 of the workbook was generated from Sheet 2, a 25-row rate card keyed by property Type and Grade. Both columns are therefore present as input features, and the model is being asked to predict a value that is a deterministic function of two columns it can see.

The script check_leakage.py verifies this directly. Against the 25-row lookup, all 2,130 rows match, a 100 percent match rate, across 25 distinct Type and Grade pairs that produce 18 distinct price values.

The consequence is that an R^2 of 1.0000 on the Decision Tree reflects rate-card recovery, not predictive accuracy. A decision tree that memorises twenty-five lookups is doing exactly what it appears to do, and nothing more.

The project therefore treats the model as a price estimator and labels it as such in the user interface. It records the verdict “target is derived not observed” inside metrics.json so that the caveat travels with the artefact, and documents it in the backend README. In addition, compare_leakage.py trains a leakage-free variant so that the difference can be shown side by side rather than merely asserted.

This is not a defect that can be engineered away while keeping the same target and the same features. It is a property of the dataset, and the correct response is disclosure, which is what is provided here.

6. Itinerary Generation Engine

The module itinerary.py turns a planner form into a complete day-by-day trip. It is deterministic by construction: it uses no randomness, no external calls and no model.

6.1 Route Selection

Districts are ordered geographically so that taking the first N yields a sensible trip with no backtracking. One of four route lists is selected by trip length. If a trip outlasts its unique districts, the extra days are given to the richest stops in place rather than looping back to the start.

Table 7: Itinerary Routes by Trip Length

Length (days)	Route list	Districts
1 to 4	Highlights	4
5 to 8	Classic	8
9 to 14	Grand tour	15
15 and above	Grand plus extensions	25

6.2 Attraction Selection

A curated catalogue of 103 attractions is keyed by district. Each attraction is tagged with the wizard's interest categories and carries a suggested time and an activity category. Selection scores each attraction by its overlap with the chosen interests, plus a baseline for evergreen culture, nature and beach content, and then orders the day by clock time rather than by score.

Across a multi-night stop, names already shown are remembered, so a two-night stay presents new sights on its second day instead of repeating itself. Districts without a catalogue entry fall back to a generic day.

6.3 Stay Selection

Overnight stays are sampled from the real dataset, filtered by district, budget band, preferred property type and the number of rooms the party needs. Rooms are derived from party size because dataset rates are quoted per double occupancy.

For a luxury budget, the pool is ranked by grade and then price. Otherwise, the property closest to the band's target nightly rate is chosen, with ties broken by grade and room count. Because the chosen property carries its published rate, the accommodation figure in the budget is real rather than modelled.

6.4 Budget Calculation

The breakdown has four lines. Accommodation is the sum of actual stay rates multiplied by rooms. Food, transport and activities are per-person allowances for the budget band, scaled down slightly on longer trips as fixed costs are spread out. Only the accommodation line is a quoted price; the other three are planning allowances, and the interface states this.

6.5 The Free-Text Notes Box

The wizard offers an optional free-text box labelled "Anything else", which invites dietary needs, must-see places and mobility notes.

No natural language processing model is used, and none could be. Training one requires labelled text, and this project has none. The only free-text column in the dataset is Remarks, which holds the same constant in all 2,130 rows and therefore carries no information. The destination catalogue holds nine hand-written blurbs with no labels. None of nltk, spacy, transformers or torch is installed, and the approved stack is scikit-learn with tree ensembles.

The field is instead handled by deterministic keyword rules, consistent with the rest of the planner: a given sentence always produces the same output, there is no model file to version, and nothing can drift between training and serving.

Three rule families are matched against the lowercased text. A longest-first index of district and attraction names resolves place mentions, so that a longer phrase such as “Galle Fort ramparts” is considered before the district “Galle” that it contains.

Table 8: Notes Keyword Rule Families

Family	Example triggers	Count
Dietary	vegan, halal, gluten	12
Access	wheelchair, elderly, pram	16
Highlights	photography, honeymoon	10

The engine produces three kinds of output:

- Dietary, access and highlight signals are reported in a summary and repeated on every day as a reminder, so they travel with the plan.
- A named attraction whose district is on the route is pinned into that day’s activities and marked as requested.
- A mention whose district is outside the route is reported as “not on this route” rather than silently rerouting, because the route must remain a function of trip length alone.

The raw text, the parsed signals and the summary are all stored with the trip and returned by the read endpoint, so the interface echoes the user’s own words back instead of re-parsing prose on the client.

Worked example. Consider a seven-day trip with the notes “vegetarian meals please, and my father uses a wheelchair, we would love to see Kandy, and please fit in the Minneriya jeep safari”. The engine detects one dietary signal and two access signals, confirms that Kandy is already on the route, and pins the Minneriya jeep safari into day three in Polonnaruwa at its catalogue time of 14:00.

7. API Reference

The API is served at http://127.0.0.1:5000 with JSON in and JSON out. Its twenty-five endpoints are grouped below by resource.

7.1 Dataset and Model

Table 9: Dataset and Model Endpoints

Method	Path	Purpose
GET	/api/health	Rows, models, db counts
GET	/api/metrics	Metrics and importances
GET	/api/filters	Districts, types, price range
GET	/api/accommodations	Search, filter, paginate
GET	/api/accommodations/:id	Detail plus estimate
GET	/api/registry	Search the establishment registry
GET	/api/price-list	Sheet 2 rate card
POST	/api/predict	Estimate a nightly price

7.2 Authentication

Table 10: Authentication Endpoints

Method	Path	Auth	Purpose
POST	/api/auth/register	none	Create an account
POST	/api/auth/login	none	Sign in
POST	/api/auth/demo	none	Sign in as demo
GET	/api/auth/me	any	Current user
PUT	/api/auth/profile	any	Update profile

7.3 Destinations

Table 11: Destination Endpoints

Method	Path	Auth	Purpose
GET	/api/destinations	none	List, search, filter
GET	/api/destinations/:id	none	One destination
POST	/api/destinations	admin	Create
PUT	/api/destinations/:id	admin	Update
DELETE	/api/destinations/:id	admin	Remove

7.4 Trips

Table 12: Trip Endpoints

Method	Path	Auth	Purpose
POST	/api/trips/plan	none	Build and save
GET	/api/trips/:id	none	Read by link
GET	/api/trips	any	Saved by account
DELETE	/api/trips/:id	owner	Remove one

The read endpoint is deliberately public so that an itinerary URL works as a shareable link. Identifiers carry forty-eight bits of random value and are therefore not guessable, but this is link-based visibility rather than a per-trip access control list. Listing and deletion are scoped to the signed-in account.

7.5 Admin

Table 13: Admin Endpoints

Method	Path	Auth	Purpose
GET	/api/admin/stats	admin	Dashboard tiles
GET	/api/admin/analytics	admin	Chart series
GET	/api/admin/users	admin	User table
DELETE	/api/admin/users/:id	admin	Remove an account

Analytics return figures derived from actual stored rows. When nothing has been created, the series come back empty and the charts show an empty state rather than fabricated sample numbers, so the dashboard cannot contradict itself.

7.6 Status Code Conventions

Table 14: API Status Code Conventions

Code	Meaning
200	Success
201	Created
400	Invalid input
401	Missing or expired token
403	Authenticated but not allowed
404	Unknown id or endpoint
409	Email already registered
503	No model loaded

One deliberate and non-obvious rule applies: a wrong password returns 400, not 401. The frontend client treats any 401 as proof that the session has ended and performs a full-page redirect to the login screen, which would swallow the error message before the user could read it. Returning 400 keeps the failure on the page where it belongs. Unauthenticated access still returns 401, and a signed-in user who lacks the admin role receives 403.

8. Authentication and Authorisation

8.1 Model

A single demo account is seeded on first run, and the React application signs itself in as that account on first load, so the protected pages open without a login form. The demo account carries the admin role, which makes the admin area reachable for evaluation.

Choosing Logout switches automatic sign-in off by writing a flag to local storage. The login form then becomes reachable, and the printed demo credentials are used.

8.2 Tokens

Tokens are HMAC-SHA256 signatures over a user identifier and an expiry time, with the key stored in a file under the data directory. Verification needs no database lookup and no session table. Token lifetime is thirty days.

8.3 Passwords and Guards

Passwords are hashed with Werkzeug’s hashing utilities rather than stored in plain text or merely obfuscated. Two route guards, one for any signed-in user and one for the admin role, are applied at the route layer so that authorisation cannot be forgotten in an individual handler.

8.4 Security Scope

The security implementation is of demonstration grade, and the documentation states this rather than implying production readiness. There is no rate limiting, no email verification and no refresh token, and cross-origin access is unrestricted, which is appropriate for local development and unsuitable for the internet. The seeded account has a published password. These constraints are restated in Section 12.

9. Frontend Implementation

9.1 Routes

Table 15: Frontend Routes

Route	Page	Access
/	Home, featured	Public
/accommodation	Dataset grid and map	Public
/accommodation/:id	Property detail	Public
/destinations	Destination grid	Public
/destinations/:id	Destination detail	Public
/hotels	Booking.com search	Public
/hotels/:id	Hotel detail	Public
/registry	Establishment registry	Public
/registry/:id	Registry detail	Public
/plan-trip	Two-step wizard	Public
/itinerary/:id	Generated plan	Public
/login	Sign in	Signed out only
/register	Sign up	Signed out only
/profile	Account settings	Protected
/my-trips	Saved itineraries	Protected
/my-trips/:id	Trip detail	Protected
/admin	Dashboard	Admin
/admin/destinations	Catalogue CRUD	Admin
/admin/users	User table	Admin
/admin/analytics	Charts	Admin

9.2 Service Layer

Two separate axios clients exist, and the reason is worth recording. The general client attaches the bearer token and, on receiving a 401, clears the token and performs a full-page redirect to the login route. That behaviour is correct for authenticated pages but destructive for public ones, because a public endpoint answering 401 would log the user out of the entire application. Public dataset endpoints therefore use a second client with no such interceptor.

9.3 Rendering Notes

Map imagery uses OpenStreetMap tiles resolved from coordinates, with a styled fallback panel when a property has no usable coordinates. Tile loading is verified by measuring the rendered image width rather than the browser's load flag, which reports true even for a broken image.

Hero statistics on the home page are static placeholders and are not derived from the dataset. This is noted so that they are not mistaken for live figures.

10. External Integrations

10.1 Booking.com via RapidAPI

Hotel search uses the Booking.com DataCrawler API on RapidAPI. The key is supplied through an environment file that is excluded from version control.

The integration implements search and detail only. It performs no booking and handles no payment; each result hands off to Booking.com through an affiliate URL, which keeps the project out of the payment and cancellation surface entirely. When the key is absent, the pages render an explanatory setup panel rather than failing, which is how the application currently behaves.

10.2 OpenStreetMap

Map tiles and coordinates come from OpenStreetMap, with attribution rendered on the map. No API key is required.

11. Testing and Verification

Table 16: Test Suite Results

Suite	Scope	Result
test_api.py	HTTP surface	97 passed, 0 failed
test_itinerary.py	Planner engine	All checks passed
eslint / vite build	Frontend	Clean, builds

The suite test_api.py exercises every endpoint, including the failure paths, and cleans up after itself, so running it does not grow the user, destination or trip tables. Its assertions cover the field names the interface actually reads, which caught a class of bug in which the database held the right data under snake_case names while the interface expected camelCase aliases.

The notes rules have their own assertions covering:

- detection of dietary, access and highlight signals;
- pinning of named attractions;
- the guarantee that a pinned attraction is never duplicated within its day;
- honest reporting of off-route mentions;
- the absence of false signals from empty or unrecognised prose;
- survival of the database round trip; and
- determinism of identical input.

Beyond the automated suites, the application was exercised in a browser across every route listed in Section 9.1. This included the full planner flow from the wizard through the generated itinerary to the saved trip and its deletion, with no console errors observed.

12. Known Limitations

Table 17: Known Limitations

#	Limitation
1	**Prices are derived, not observed.** Disclosed in full in Section 5.5. The headline accuracy must never be presented as predictive performance.
2	**The dataset is held in memory.** Mutable state is in SQLite, but the 2,130-row table is loaded into pandas at startup. That is appropriate for a read-only table of this size but is not how a production deployment would serve it; the store module is the single point that would change.
3	**The development server is not a production server.** The framework's built-in server is explicitly not intended for production traffic. A production deployment would place a proper WSGI server in front and serve the built frontend as static files.
4	**Authentication is of demonstration grade.** Passwords are properly hashed and tokens are properly signed, but there is no rate limiting, no email verification and no refresh token, and the seeded administrator account has a published password. This build must not be exposed to the internet as it stands.
5	**Coordinates are imputed for approximately 760 rows.** District medians are used, so some map markers are approximate rather than exact.
6	**Ten duplicate property names exist in the source data.** They are deliberately retained so that the fitted price distribution is not altered.
7	**Trip visibility is link-based.** The read endpoint is public by design so that itinerary URLs are shareable; listing and deletion remain scoped to the owner.
8	**The Booking.com key is not configured.** The hotel pages therefore render their setup panel. Supplying the key activates them with no code change.

13. Deployment and Runbook

13.1 Starting the System

Two processes are required. From the backend directory, run the training step once if the model artefacts are absent, and then start the API. From the frontend directory, start the development server. The frontend proxies API calls, so no cross-origin configuration is needed in development.

On its first run, the API creates the database file, seeds the destination catalogue and the demo account, and prints the row counts and the demo credentials.

13.2 Seeded Credentials

The demo account is printed at startup and when the store module is run directly. It signs in automatically on first page load; choosing Logout makes the login form reachable, after which these credentials are entered manually.

13.3 Regenerating Data and Models

The import script converts a fresh copy of the source workbook into the two CSV files and is the reproducible path from source to data. The training script then rebuilds every artefact and prints the leakage report, so a new workbook can be imported and the models retrained without any manual steps.

13.4 Running the Tests

Both suites are run directly with the Python interpreter from the backend directory. The API suite reports its pass and fail counts and exits with a non-zero status on failure, so it can be wired into a build step unchanged.

13.5 Directory Layout

The repository holds the backend, the frontend, a docs directory containing this document and the proposal together with their generator, and a prototype directory. The backend is organised as data (the dataset, database and seed file), models (the trained artefacts) and the Python modules described in Table 1.

Appendix A: Summary of Design Decisions

Design decisions and rationale

Decision	Rationale
Deterministic planner	Plans must be regenerable and shareable
Two datasets, kept apart	Only one has prices; the other has contacts
SQLite over MySQL	No client installed; stdlib needs no server
Blueprints per resource	Avoids a single oversized module
Dataset via app config	Prevents a circular import
HMAC tokens, no sessions	Verification without a database hit
400 not 401 on bad password	The client redirects on any 401
Keyword rules over NLP	No labelled text exists to train on
Notes stored verbatim	The user's own words are echoed back
Off-route mentions reported	The route stays a function of duration
Leakage disclosed, not hidden	The figure is rate-card recovery
Separate public axios client	A public 401 must not end the session
Duplicate names retained	Keeps the fitted distribution honest

Appendix B: Sample Generated Trip

The following is the shape returned by the trip endpoints, abridged to one day.


```
{
 "title": "Colombo to Galle: 7 days",
 "route": "Colombo -> Kandy -> Galle",
 "duration": 7,
 "interestLabels": ["Beaches", "Culture & history"],
 "startDate": "2026-12-10",
 "budgetType": "mid-range",
 "groupSize": 2,
 "rooms": 1,
 "nights": 6,
 "districts": ["Colombo", "Anuradhapura", "Polonnaruwa",
             "Kandy", "Nuwara Eliya", "Badulla", "Galle"],
 "notes": "Please fit in the Minneriya jeep safari.",
 "notesParsed": {
   "places": ["Minneriya jeep safari"],
   "dietary": [], "access": [], "highlights": []
 },
 "notesSummary": [
   "Added to your plan: Minneriya jeep safari (Polonnaruwa)."
 ],
 "itinerary": [
   {
     "day": 3,
     "district": "Polonnaruwa",
     "note": null,
     "activities": [
       { "time": "09:00", "name": "Gal Vihara rock Buddhas",
         "category": "Culture" },
       { "time": "14:00", "name": "Minneriya jeep safari",
         "category": "Wildlife", "pinned": true }
     ],
     "stay": { "name": "GALKADAWALA FOREST LODGE",
             "double": 12500.0, "meal_plan": "BB" }
   }
 ],
 "budget": {
   "total": 297600.0,
   "currency": "LKR",
   "breakdown": [
     { "label": "Accommodation", "value": 132000.0 },
     { "label": "Food & drink", "value": 89000.0 },
     { "label": "Transport", "value": 41000.0 },
     { "label": "Activities & entrance fees", "value": 35600.0 }
   ]
 }
}
```


Only the Accommodation line is a published rate. The remaining three are planning allowances for the selected budget band, and the interface states this wherever a total is shown.